Skill registry — 2026-05-20

Every `.claude/skills/*/SKILL.md` across the portfolio. Rows with a green left edge are **measured** from real Claude Code transcripts (90-day window); the rest are **estimated** by the skill author.

Reference: Claude Code built-ins

Claude Code's own slash commands — **not part of this portfolio**, shown for scale. **Excluded from every total below.**

Skill	Description	Cost / use	Times run	Tokens used
/review today	Claude Code built-in PR code review	1.14M (measured)	12 in 90d → ~49/yr	13.66M in window ~55.78M/yr proj.

Aggregate

Repo	Skills	Times run (measured)	Tokens used / yr
AudiobookMaker	10	4 in 90d → ~16/yr	~5.06M
Spacepotatis	14	5 in 90d → ~20/yr	~5.41M
mikkonumminen.dev	2	2 in 90d → ~8/yr	~806K
claude-skills	9	2 in 90d → ~8/yr	~87K
Total	**35**	**13 in 90d** → ~52/yr	**~11.36M**

AudiobookMaker — https://github.com/MikkoNumminen/AudiobookMaker

Skill	Description	Cost / use	Times run	Tokens used
ai-codegen-smell-audit 3d ago	Read-only audit for specific failure modes that recur in LLM-generated code — defensive guards on impossible cases, generic names in domain code, swallowed errors, single-use helpers, mirror tests, and so on. Each check has a concrete smell example and a concrete legitimate counter-example so the auditor (human + assistant) can tell signal from noise. Produces `docs/audits/ai-smell-<YYYY-MM-DD>.md` with severity-ranked findings. Use whenever the user says "smell audit", "check for AI-codegen smells", "review this branch for LLM-style sludge", "audit the code for generated-code patterns", or asks for a calibration pass before merging a large generated diff. NOT a witch hunt for "AI-written code" — every finding must be a specific testable pattern with a concrete example, not a vibe.	199K (measured)	1 in 90d → ~4/yr	199K in window ~798K/yr proj.
audit 1d ago	Run a multi-phase robustness audit of the current codebase. Phase 1 tries language-appropriate static-analysis tools, skipping cleanly if absent. Phase 2 spawns five parallel subagents across non-overlapping scopes — resource lifecycle, data integrity, concurrency, error paths, external boundaries — each returning findings with file:line citations and severity. Phase 3 aggregates into docs/audits/audit-YYYY-MM-DD.md with a severity tally (critical / high / medium / low) and a branch topology recommendation for the fix follow-up. Use whenever the user says "audit this codebase", "find bugs", "robustness review", "review for races / leaks / error swallows", "check for hidden bugs", "comprehensive review", "shake out issues before release", or asks for a defect list. Not for style / readability reviews — see the "When NOT to invoke" section below.	435K (measured)	1 in 90d → ~4/yr	435K in window ~1.74M/yr proj.
ci-failure-triage	When CI fails on a release tag or PR, walk the failure modes in the right order against a recipe library built from the project's fix(ci) commit history. Use whenever the user says "CI is red", "the build failed", "tag failed CI", "release-build broke", or any time gh run list shows a recent failure on master or a release tag.	1K (est.)	88 / yr (est.)	~88K /yr (est.)
copyright-scan 8d ago	Scan a git diff (staged by default, or a named revision range / file set) for accidental third-party copyright leaks before they land on origin. Use this skill whenever the user says "scan the diff", "copyright check", "scan for leaks", "is this safe to push", "check this before I commit", "did I leak anything?", or whenever you are about to create a commit or push in this repo. CLAUDE.md marks copyright leakage as a P0 — this skill turns the manual scan ritual into one pass. Produces a structured pass/fail report with file:line citations and, on fail, the exact remediation path (edit vs. un-stage vs. P0 scrub-from-origin).	14K (measured)	1 in 90d → ~4/yr	14K in window ~56K/yr proj.
pronunciation-corpus-add	Append a Finnish pronunciation failure (a word the Chatterbox Grandmom voice mispronounces) to the project's pronunciation corpus at docs/pronunciation_corpus_fi.md. Use this whenever the user or a tester (Turo is the canonical one) reports that Grandmom said a Finnish word wrong — e.g. "Grandmom pronounces X as Y", "add this to the corpus", "log this pronunciation bug", "the TTS messed up <word>". Also use when the user pastes a transcript of what was heard vs. what was written. This corpus is the evidence base for targeted Pass I / lexicon fixes; entries must be structured so patterns across reports are visible at a glance.	—	—	—

Skill	Description	Cost / use	Times run	Tokens used
release-bundle-audit	Audit the AudiobookMaker PyInstaller release bundle for unused dependencies, dead-code data files, and accidental ML-stack pollution; then propose spec-only fixes on a `chore/release-bundle-size` branch. Use whenever the user says "the installer is huge", "why is the bundle so big", "shake out unused deps before release", "shrink the .exe", "audit the .spec", or asks why a frozen build ships some specific package. Encodes the empirical finding that ~28% of the v3.11.x bundle was dead-code data files (ffplay.exe, AVIF support, Arabic phonemizer, ONNX training tools) plus a defense-in-depth exclude set that prevents future regressions when a contributor accidentally pip-installs torch on the build machine.	4K (est.)	4 / yr (est.)	~14K /yr (est.)
release-cut 8d ago	Cut a new AudiobookMaker release (bump APP_VERSION, tag vX.Y.Z, push, verify CI lands SHA-256 in release notes AND sidecar). Use this skill whenever the user says "cut a release", "bump the version", "ship X.Y.Z", "make a release", "release X.Y.Z", or asks to tag a new version. Auto-update is existential for this project — a release that ships without a valid SHA-256 locks every existing user out of one-click updates. This skill encodes every moving part so that can never happen again.	188K (measured)	1 in 90d → ~4/yr	188K in window ~753K/yr proj.
voice-pack-finnish	Build a Finnish voice pack from a source audio file end-to-end — chunked analyze with ECAPA diarization, transcript validation per speaker, branching to LoRA training (long sources) or few-shot ref-clip packaging (short sources), and ear-check by synth. Use whenever the user says "copy the voices", "extract voices from this clip", "make a voice pack from this audio", "I have a short Finnish clip", or hands you a Finnish podcast / interview / audiobook to mimic. Encodes the empirically-validated Finnish pipeline. CRITICAL — Finnish source only (ASR + finetune model are Finnish-specific); for English / other languages this skill does not apply. Never assume single-speaker, never trust pyannote labels without validation, never install copyright-derived packs to ~/.audiobookmaker.	7K (est.)	248 / yr (est.)	~1.61M /yr (est.)
work-session	Start, pause, or finish a TODO.md work session as one of the 4 permanent Claude sessions in AudiobookMaker. Use this whenever the user says "claim task X", "take task Y", "start working on Z", "pick something", "I'm done", "finish up", "pause", "I'm blocked", or "go idle". Also use before touching any code on a fresh session to make sure the status board and in-progress list are honest. Parallel Claudes collide without this discipline; the shared TODO.md protocol is the only way four sessions stay coherent.	2K (est.)	—	—
worktree-launch	Start a new parallel Claude session safely. Pick a free session slot (Claude 1/2/3/4), create the worktree under .claude/worktrees/<branch>, claim a task in TODO.md, verify isolation actually held after the agent runs. Use whenever the user says "start a new session", "spawn another Claude", "open a worktree", or "let me run two of you in parallel".	800 (est.)	—	—

Spacepotatis — https://github.com/MikkoNumminen/Spacepotatis

Skill	Description	Cost / use	Times run	Tokens used
ai-codegen-smell-audit	Read-only audit for 10 concrete failure modes that recur in AI-generated code (defensive checks on non-nullable types, paraphrase comments, single-use helpers, generic names in domain code, swallowed errors, mirror tests, phantom TODOs, duplicated helpers, over-typed primitives, intra-file style drift). Produces a markdown report under docs/audits/. Does not modify code. Designed for PR review and on-demand scans, not for use during initial generation.	10K (est.)	30 / yr (est.)	~300K /yr (est.)

Skill	Description	Cost / use	Times run	Tokens used
audit	Orchestrates the multi-phase modular-architecture audit + refactor. Phase 0 sets up agents; Phase 1 inventories every file; Phase 2 proposes module boundaries; Phase 3 mechanically extracts modules one at a time (parallelizable across worktrees); Phase 4 writes AI-first documentation; Phase 5 verifies. Each phase produces a written artifact under docs/audit/ and STOPS at a gate for user approval. Never auto-advances.	5K (est.)	5 / yr (est.)	~25K /yr (est.)
balance-review	Diff uncommitted changes to game data (weapons, enemies, waves, missions, perks, augments, loot pools, solar systems) and report DPS, TTK, energy-cost-per-DPS, augment-folded effective DPS, loot-pool roster shifts, and drop-rate deltas vs HEAD.	14K (est.)	50 / yr (est.)	~675K /yr (est.)
content-audit 17d ago	Pre-commit content invariants check — orphan refs (enemy / weapon / sprite / pod / loot-pool / mission system / story trigger), missing sprite generators, perk drop-weight sanity, mission prereq DAG, story integrity (voice + music files, trigger refs), storyTriggers helper coverage.	53K (measured)	1 in 90d → ~4/yr	53K in window ~213K/yr proj.
equipment 17d ago	Create, modify, or remove a weapon or piece of equipment (augment / reactor / shield / armor) — including visual changes to how it appears in combat or in the loadout UI. Covers stats, sprites, prices, and the full CRUD lifecycle for the entire ship-loadout content surface.	401K (measured)	2 in 90d → ~8/yr	803K in window ~3.21M/yr proj.
new-enemy	Scaffold a new enemy — enemies.json entry, BootScene placeholder sprite generator, optional test wave, and integrity test verification.	6K (est.)	25 / yr (est.)	~138K /yr (est.)
new-migration	Add a Postgres schema migration end-to-end — dated SQL file in db/migrations/, Database interface update in src/lib/db.ts, prod application via scripts/migrate.mjs, schema verification via scripts/check-schema.mjs, and the PR-body checkbox that gates merge. Enforces CLAUDE.md §7a HARD RULE so the next save POST doesn't 500.	7K (est.)	15 / yr (est.)	~105K /yr (est.)
new-mission	Scaffold a new combat mission across missions.json, waves.json, the galaxy planet binding, and a smoke test — in one shot.	8K (est.)	30 / yr (est.)	~240K /yr (est.)
new-perk	Scaffold a new mission-only perk — perks.ts entry, BootScene icon generator, HUD chip wiring, and (for actives) a switch case in PerkController.apply / triggerActive.	9K (est.)	10 / yr (est.)	~90K /yr (est.)
new-solar-system	Add a new solar system to the galaxy — solarSystems.json entry (incl. galaxy bed), SolarSystemId union extension, optional unlock gating, REQUIRED on-system-enter cinematic (voice + music) AND dedicated galaxy-view music bed, and mission-binding TODO list.	13K (est.)	5 / yr (est.)	~65K /yr (est.)
new-story 20d ago	Add, modify, or remove story content — cinematic popups, voiceovers, music beds, body text, written synopses, and auto-trigger wiring. Covers the full CRUD lifecycle for the Story log.	56K (measured)	1 in 90d → ~4/yr	56K in window ~222K/yr proj.
new-weapon *(redirect)*	Superseded by /equipment. Adding a weapon now lives inside the broader CRUD-and-visuals skill. This stub redirects.	—	—	—
save-roundtrip-audit 17d ago	Pre-commit save-pipeline invariants check — walks every StateSnapshot field through the 8 layers of the save round-trip (snapshot interface → toSnapshot → POST schema → POST handler → DB column → migration → GET handler → RemoteSaveSchema → sync.loadSave → hydrate) and flags any layer that silently drops the field. Read-only.	19K (measured)	1 in 90d → ~4/yr	19K in window ~74K/yr proj.

Skill	Description	Cost / use	Times run	Tokens used
security-audit	Orchestrates the multi-phase security audit + remediation. Phase 0 sets up agents; Phase 1 maps the full attack surface (entry points, auth, authz, input, secrets, data exposure, deps, transport, client, ops); Phase 2 turns findings into a prioritized remediation plan; Phase 3 fixes one finding at a time with regression tests (crit/high sequential, low/med parallelizable in worktrees); Phase 4 writes AI-first security documentation (SECURITY.md, threat model, invariants, code-level markers, lint rules); Phase 5 verifies. Each phase produces an artifact under docs/security/ and STOPS at a gate for user approval. Never auto-advances. Critical findings surface immediately.	5K (est.)	10 / yr (est.)	~50K /yr (est.)

mikkonumminen.dev — https://github.com/MikkoNumminen/mikkonumminen.dev

Skill	Description	Cost / use	Times run	Tokens used
skill-registry 1d ago	Scan every sibling repo under D:/koodaamista for `.claude/skills/*/SKILL.md` files and emit a consolidated registry as a structured JSON document — per-skill name, description, redirect flag, and (where receipts exist) token-savings estimates. Runs one Sonnet sub-agent per repo in parallel for swiftness. Output goes to a dated JSON file under `.claude/agent-verdicts/`, committed so other Claude sessions can read the current inventory without re-running.	83K (measured)	1 in 90d → ~4/yr	83K in window ~331K/yr proj.
sync-readmes 2d ago	Audit project data against sibling repos' READMEs and open a PR with drift corrections. Runs parallel Sonnet diff agents (one per sibling repo), synthesizes drift, applies en+fi+sv corrections plus tech-list updates, and lands a PR for review on GitHub.	119K (measured)	1 in 90d → ~4/yr	119K in window ~475K/yr proj.

claude-skills — https://github.com/MikkoNumminen/claude-skills

Skill	Description	Cost / use	Times run	Tokens used
ai-codegen-smell-audit	Read-only audit for specific failure modes that recur in LLM-generated code — defensive guards on impossible cases, generic names in domain code, swallowed errors, single-use helpers, mirror tests, and so on. Each check has a concrete smell example and a concrete legitimate counter-example so the auditor (human + assistant) can tell signal from noise. Produces `docs/audits/ai-smell-<YYYY-MM-DD>.md` with severity-ranked findings. Use whenever the user says "smell audit", "check for AI-codegen smells", "review this branch for LLM-style sludge", "audit the code for generated-code patterns", or asks for a calibration pass before merging a large generated diff. NOT a witch hunt for "AI-written code" — every finding must be a specific testable pattern with a concrete example, not a vibe.	—	—	—
audit	Run a multi-phase robustness audit of the current codebase. Phase 1 tries language-appropriate static-analysis tools, skipping cleanly if absent. Phase 2 spawns five parallel subagents across non-overlapping scopes — resource lifecycle, data integrity, concurrency, error paths, external boundaries — each returning findings with file:line citations and severity. Phase 3 aggregates into docs/audits/audit-YYYY-MM-DD.md with a severity tally (critical / high / medium / low) and a branch topology recommendation for the fix follow-up. Use whenever the user says "audit this codebase", "find bugs", "robustness review", "review for races / leaks / error swallows", "check for hidden bugs", "comprehensive review", "shake out issues before release", or asks for a defect list. Not for style / readability reviews — see the "When NOT to invoke" section below.	—	—	—

Skill	Description	Cost / use	Times run	Tokens used
mikko-audit-suite	Run every `mikko-*` audit skill that's relevant to the current codebase, in a sensible order, and produce one index document linking to each audit's report. Detects the codebase shape (language, framework, security surface), maps to the suggested audit list (same logic as `/mikko-help --detect`), confirms once with the human, then runs each audit sequentially. Use whenever the user says "run all audits", "full audit", "audit everything", "what's wrong with this codebase", or before a major release. Expensive — total tokens are the sum of every dispatched audit (typically ~100-500K). The orchestrator itself is cheap; the dispatched audits are what cost.	—	—	—
mikko-help today	List every installed `mikko-*` skill with its description (or with its `barney:` field when `--barney` is passed). The fast answer to "I know I have a skill for this but can't remember which one" / "what mikko skills do I have" / "list my skills" / "remind me what's installed." Pass `--detect` for codebase-aware audit recommendations ("which audit should I run on this repo" / "what's relevant for this codebase"). Pass `--barney` for plain-English short descriptions ("give me the friendly version" / "plain-English list"). `--detect` and `--barney` combine independently. No sub-agents, no network, no measurements — just a table. Full mode docs live in the body's "Flags" section.	17K (measured)	1 in 90d → ~4/yr	17K in window ~67K/yr proj.
mikko-install	Install, update, list, or uninstall `mikko-*` skills from the cloned `claude-skills` source repo into the user-wide skill directory (`~/.claude/skills/mikko-*`). Wraps the existing `install-mikko.sh` and `install.sh` scripts so the install loop is reachable from inside any Claude Code session via a slash command — no need to leave the conversation, find the repo, and run bash by hand. Use whenever the user says "install the mikko skills", "install everything", "add mikko-readme-drift-sync", "update my mikko skills", "what's new since last install", or "uninstall mikko-foo". Locates the source repo via the current working directory, parent directories, or known sibling locations (configurable). Does NOT clone the repo (the user clones it once, by hand). Does NOT touch skills outside the `mikko-*` prefix.	—	—	—
react-anti-patterns-audit	Read-only audit for six concrete React anti-patterns that recur in component code (key=index in lists, useEffect for derived state, dep-array lies, state mutation instead of replace, missing effect cleanup, multiple sources of truth). Each check has a concrete smell example and a concrete legitimate counter-example so the auditor can tell signal from noise. **Runs a pre-flight check first** — if the target codebase isn't a React project, the skill aborts cleanly with a suggestion to use a different skill. Use whenever the user says "audit my React code", "review this for React anti-patterns", "check my hooks", "find re-render issues", or before merging a substantial component PR. NOT a witch hunt for "non-idiomatic React" — every finding must be a specific testable pattern with a concrete example.	—	—	—
readme-drift-sync	Detect drift between what the repo actually contains and what `README.md` claims, then rewrite ONLY the drifted sections in the README's existing voice. Five drift axes (file structure, dependencies, skills, features, status). Voice-preserving — the skill extracts a voice profile from the existing README first and any rewrite must read as if the original author wrote it. **Write-with-approval**: emits a working-tree edit to `README.md` plus a drift report and a voice-profile cache, then STOPS. Does NOT commit, does NOT push, does NOT regenerate the README from scratch. Use whenever the user says "sync the README", "update the README", "check README drift", "has the README fallen out of date", or after a release-cut before the next push. NOT for initial README creation, full rewrites, CONTRIBUTING/CHANGELOG edits, or general code edits where the README isn't in scope.	—	—	—

Skill	Description	Cost / use	Times run	Tokens used
security-audit	Orchestrates the multi-phase security audit + remediation. Phase 0 sets up agents; Phase 1 maps the full attack surface (entry points, auth, authz, input, secrets, data exposure, deps, transport, client, ops); Phase 2 turns findings into a prioritized remediation plan; Phase 3 fixes one finding at a time with regression tests (crit/high sequential, low/med parallelizable in worktrees); Phase 4 writes AI-first security documentation (SECURITY.md, threat model, invariants, code-level markers, lint rules); Phase 5 verifies. Each phase produces an artifact under docs/security/ and STOPS at a gate for user approval. Never auto-advances. Critical findings surface immediately.	—	—	—
skill-usage today	Measure actual Claude Code skill usage from local transcript JSONL files. Walks `~/.claude/projects/*/*.jsonl` (and their `subagents/*.jsonl` sidechain files), filters assistant messages by the harness-emitted `attributionSkill` field, groups invocations by `(sessionId, skill)`, sums tokens deduped by `requestId`. Emits a dated `SKILL-USAGE-{YYYY-MM-DD}.json` with per-skill measured invocation counts and token totals. Designed to replace the editorial estimates in skill catalogs (`docs/SKILLS.md`, registry verdicts) with real receipts.	5K (measured)	1 in 90d → ~4/yr	5K in window ~20K/yr proj.

Generated 2026-05-20 at 04:34 UTC. Measured window: 13 invocations across custom + built-in rows · 1.99M tokens in 90d.